SOFTWARE REQUIREMENTS SPECIFICATION

Weather Station Dashboard

SRS Document and UML Diagrams

Project: Weather Station Dashboard
Purpose: Retrieve and parse weather API data and display current conditions and forecasts.

Prepared as an academic project document

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements for a Weather Station Dashboard. The application retrieves weather information from an external weather API, parses the response, and presents useful weather details to the user.

1.2 Scope

The dashboard allows a user to search for a city or location, view current weather conditions, view a forecast, and refresh the displayed data. The system depends on an internet connection and a weather service API.

1.3 Intended Users

General users who need quick access to local or selected-location weather information; developers and testers who maintain or verify the application.

2. Overall Description

2.1 Product Perspective

The dashboard is a client application connected to a third-party Weather API through an API service module. The API returns data, commonly in JSON format, which the application validates and converts into display-ready weather information.

2.2 Operating Environment

A modern web browser or mobile application environment, internet access, and a configured weather API endpoint/API key. Exact technologies may be selected during implementation.

2.3 Assumptions and Dependencies

The weather API is available and provides valid data; the API key is configured securely; the user enters a valid location; and the device has network access.

3. Functional Requirements

ID	Requirement	Description
FR-01	Search Location	The system shall allow the user to enter a city or location.
FR-02	Retrieve Current Weather	The system shall request current weather information from the weather API.
FR-03	Parse API Response	The system shall parse the API response and extract temperature, condition, humidity, wind speed, and other available fields.
FR-04	Display Weather	The system shall display the current weather in a clear dashboard layout.
FR-05	Show Forecast	The system shall display forecast information when supported by the API.
FR-06	Refresh Data	The system shall allow the user to request updated weather data.
FR-07	Handle Errors	The system shall show a useful message when a location is invalid, the network fails, or the API returns an error.
FR-08	Show Update Status	The system should indicate loading status and the time of the latest successful update.

4. Non-Functional Requirements

Performance: The dashboard should show results promptly under normal network conditions and display a loading indicator while waiting for the API.

Reliability: The application should handle missing fields, malformed responses, API timeouts, and unavailable services without crashing.

Security: API keys must not be exposed in public client code where avoidable; use HTTPS and avoid logging secrets.

Usability: Weather details should be readable, clearly labelled, and usable on common screen sizes.

Maintainability: Keep UI, API communication, and response parsing in separate modules where practical.

5. External Interface Requirements

User interface: Search box, search/refresh controls, current temperature, condition icon/text, humidity, wind speed, location name, update time, and forecast section.

Software interface: Weather API endpoint accessed through HTTPS; JSON response parser; optional location/geocoding service if required.

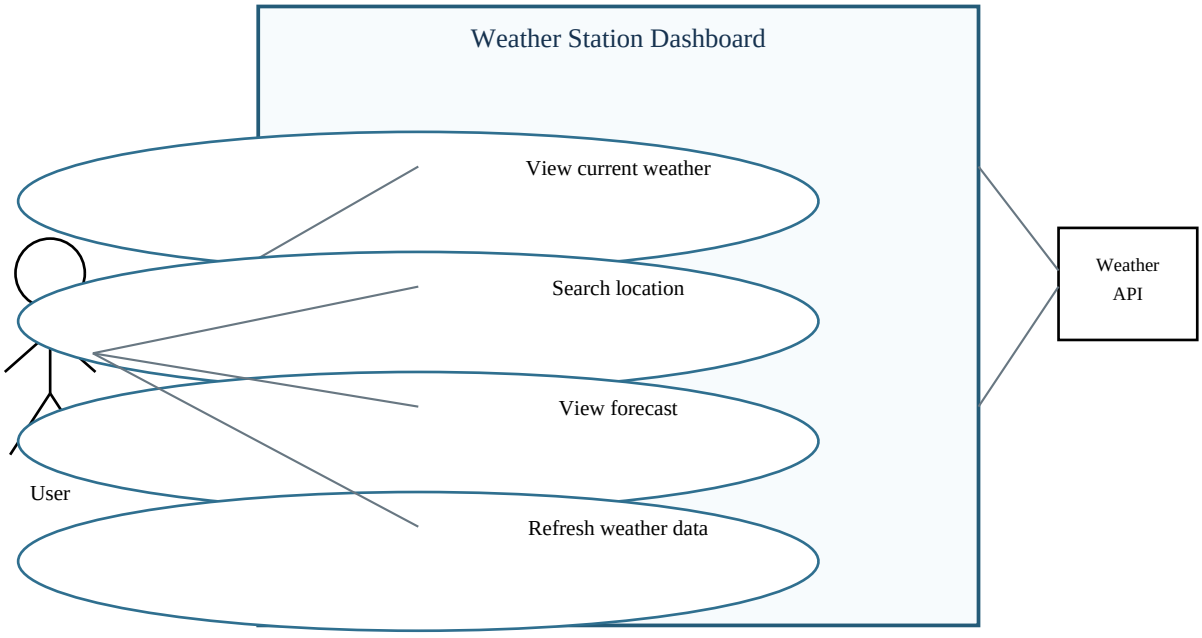
Communication interface: HTTP request/response over an internet connection.

6. Basic Use-Case Flow

1. User opens the dashboard. 2. User enters a location. 3. Dashboard sends a request to the API service. 4. API service retrieves the weather response. 5. The response is parsed and validated. 6. Dashboard displays current weather and forecast. 7. If an error occurs, the dashboard displays an appropriate message.

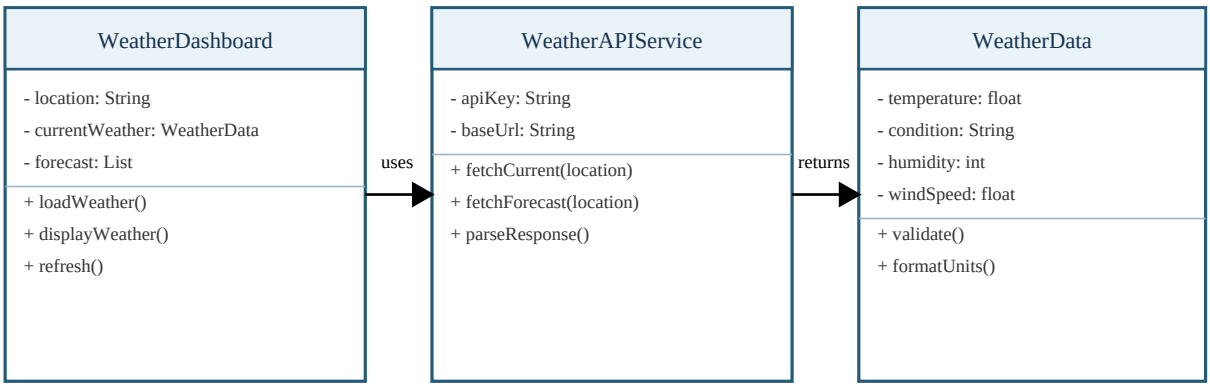
7. UML Diagram — Use Case

The main actor is the User. The Weather API is an external service used to retrieve weather data.



8. UML Diagram — Class Diagram

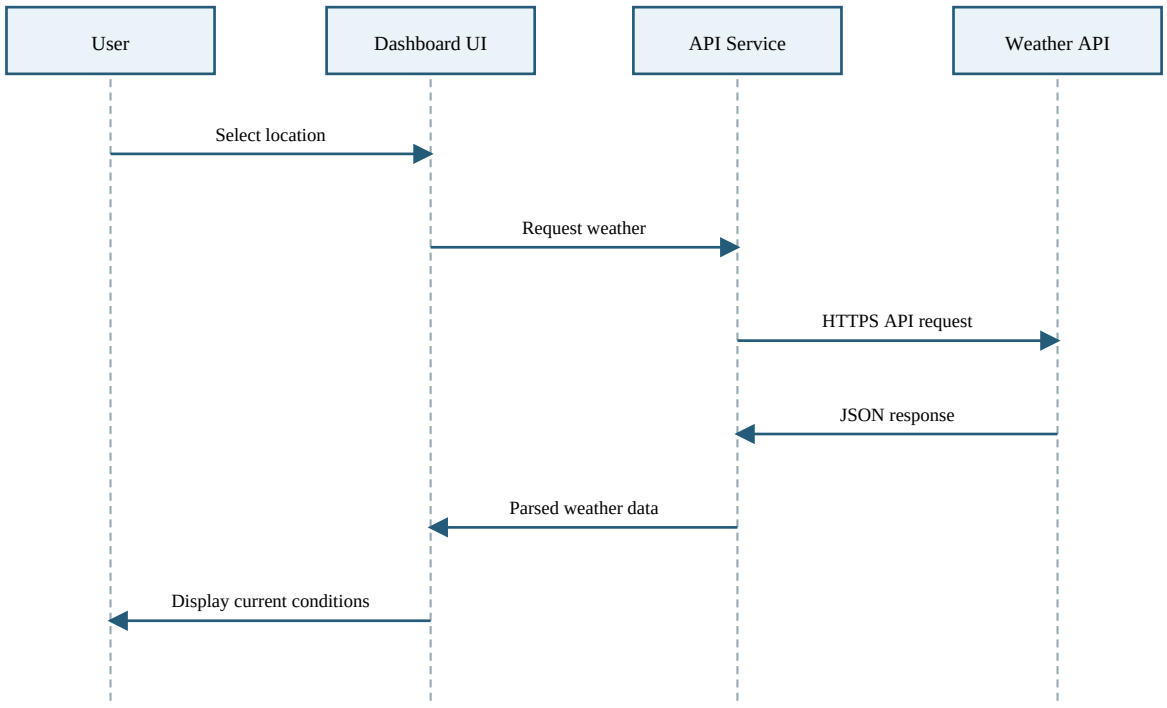
The dashboard handles user interaction, the API service retrieves and parses data, and the WeatherData model stores the weather values used by the interface.



Simplified UML class diagram

9. UML Diagram — Sequence Diagram

This sequence shows the normal flow for retrieving and displaying weather data.



10. Data Model

Field	Type (example)	Description
location	String	City or location requested by the user
temperature	Number	Current temperature in configured units
condition	String	Short weather description, e.g. cloudy
humidity	Integer	Relative humidity percentage
windSpeed	Number	Wind speed in configured units
forecast	List / Array	Forecast entries returned by the API
lastUpdated	Date-Time	Time the data was last updated

11. Error Handling

If the user enters an unknown location, show “Location not found.” If the network is unavailable, show “Unable to connect; check your internet connection.” If the API is unavailable or returns invalid data, show a friendly error and allow the user to retry. Do not display stale data as current without clearly identifying it.

12. Acceptance Criteria

• A user can search for a valid location. • Current weather values are retrieved and displayed. • The system can parse a valid API response. • Forecast information appears when available. • Refresh updates the weather. • Invalid locations and connection/API errors produce understandable messages.

13. Conclusion

The Weather Station Dashboard provides a simple way to retrieve, parse, and display weather information. The SRS defines the core requirements, while the UML diagrams describe the main actors, software classes, and request sequence.

Note: This is a general academic SRS template. Replace example fields and API details with the exact requirements of your implementation.